Design and Development of a Scalable Framework for Real-Time Multiplayer Gaming Using Unity and Networking

Subasri V
Department of CSE
Rajalakshmi Engineering College

Suganya S
Department of CSE
Rajalakshmi Engineering College

Sushma S
Department of CSE
Rajalakshmi Engineering College

Yashvinthini R
Department of CSE
Rajalakshmi Engineering College

ABSTRACT

An online multiplayer gaming platform is developed using Unity, Unity Gaming Services, and Netcode for GameObjects to support real-time interaction between multiple players. The platform provides a common structure for user authentication, multiplayer session creation and joining, networked player management, ownership handling, input processing, and synchronization. A 2D multiplayer platformer is implemented as the initial game mode, and a 3D multiplayer racing game is added as an additional game module within the same platform.

The 2D game supports networked player movement and multiplayer gameplay, while the 3D racing module introduces a dedicated racing environment, networked vehicle spawning, player-specific vehicle ownership, server-authoritative vehicle movement, synchronized race control, checkpoint validation, and multi-lap progression. The racing module uses a server-controlled race countdown to synchronize the starting point for all connected players. Player input is transmitted to the server, where vehicle movement and physics are processed before the resulting position and rotation are synchronized with other clients.

The racing system uses ordered checkpoints to validate player progress and prevent checkpoint skipping. Each vehicle maintains its own checkpoint and lap state, allowing multiple players to progress through the race independently. The modular structure separates common multiplayer services from game-specific functionality, allowing additional game modes to be integrated without redesigning the complete networking system.

CCS CONCEPTS

• Applied computing → Computer games;
• Networks → Network protocols; Network experimentation;
• Software and its engineering → Software prototyping;
• Computing methodologies → Computer graphics;
• Human-centered computing → User interface design.

KEYWORDS

Online Multiplayer Gaming, Unity, Unity Gaming Services, Netcode for GameObjects, Real-Time Networking, Network Synchronization, Game Development.

1 INTRODUCTION

Multiplayer games require multiple players to interact within the same game environment while maintaining consistent player states across connected systems. Player movement, actions, ownership, session management, and game events must be synchronized so that each connected player receives an updated view of the shared environment. A multiplayer system therefore requires both game-specific logic and a reliable networking structure for managing communication between players.

The developed platform provides a common multiplayer structure using Unity, Unity Gaming Services, and Netcode for GameObjects. The common system manages user authentication, multiplayer session creation and joining, network connections, player spawning, ownership, input handling, and real-time synchronization. This common structure allows different games to use the same basic multiplayer infrastructure instead of implementing a separate networking system for every game mode.

The platform currently contains two game modules. The first module is a 2D multiplayer platformer in which connected players are represented by networked player objects and their movement and gameplay actions are synchronized across the session. The second module extends the platform with a 3D multiplayer racing game. The racing module uses a separate racing scene containing the track, networked vehicles, spawn points, checkpoints, and race management logic.

The 3D racing module introduces additional networking and gameplay requirements. Each connected player receives an individually owned racing vehicle. Vehicle input is sent from the client to the server, while the server controls the vehicle movement and physics. The resulting vehicle transform is synchronized to the other connected players. This server-authoritative approach separates input collection from movement simulation and network synchronization.

The racing module also implements a synchronized race countdown, pre-race movement restriction, ordered checkpoint detection, and multi-lap progression. Each vehicle maintains its

own race progress, allowing different players to reach checkpoints and complete laps at different times. The current implementation supports three racing vehicles, three checkpoints per lap, and a five-lap race configuration.

The overall design separates common multiplayer services from game-specific components. Authentication, sessions, network connections, and player ownership form the shared platform layer, while movement, combat, racing, checkpoints, laps, and other gameplay rules are handled by individual game modules. This structure allows additional multiplayer games to be integrated into the platform while maintaining the existing multiplayer infrastructure.

2 RELATED WORK

Online multiplayer games are distributed interactive applications in which multiple players interact with a common virtual environment through a network. Research in this area has primarily focused on distributed game architectures, state synchronization, network latency, consistency, fairness, and scalability. These factors directly influence the responsiveness and reliability of multiplayer gameplay.

Gautier and Diot presented MiMaze, a distributed multiplayer game designed for Internet-based gameplay. The system used an IP multicast communication architecture based on RTP/UDP/IP along with distributed synchronization mechanisms to maintain consistency between participating players. Their work demonstrated the feasibility of distributed multiplayer gaming over a best-effort network and highlighted the importance of synchronization in maintaining a consistent shared game state [1].

Lin, Guo, and Paul proposed Sync-MS, a synchronized messaging service for real-time multiplayer distributed games. Their approach addressed fairness problems caused by differences in network delay between players. The proposed Sync-out and Sync-in mechanisms were designed to synchronize state updates and player actions while balancing fairness against response time [2].

Jiang, Safaei, and Boustead investigated the major challenges associated with latency and scalability in networked games. Their survey identified network latency as an important factor affecting real-time interaction and discussed scalability techniques required to support increasing numbers of players and network traffic [3].

Moon, Muthukkumarasamy, Nguyen, and Kim studied consistency maintenance in distributed network games. Their work examined conservative and optimistic approaches for maintaining consistency. The study also discussed the trade-off between latency and consistency and investigated synchronization approaches for distributed multiplayer environments [4].

Mauve, Vogel, Hilt, and Effelsberg investigated consistency for replicated continuous applications, including networked computer games and distributed virtual environments. Their work introduced local-lag and timewarp concepts for maintaining consistency and correctness when replicated application states are affected by user actions and network delays [5].

Pantel and Wolf examined the impact of network transmission delay on real-time multiplayer games. Their study highlighted that communication delay can produce differences in the perceived game state between players, making latency an important consideration in multiplayer game design [6].

Ahn, Cheng, Baek, and Fisher proposed a multiplayer real-time game protocol architecture aimed at reducing network latency and improving game-server efficiency. Their work considered the soft real-time characteristics of multiplayer games and proposed scheduling and spatial-domain techniques for handling client events and reducing queuing delays [7].

Bharambe, Pang, and Seshan developed Colyseus, a distributed architecture for online multiplayer games. The architecture exploited predictable game workloads and tolerance for weakly consistent state to distribute game traffic and computation. Their evaluation demonstrated that distributed architectures can improve scalability while maintaining low-latency gameplay [8].

Fujita investigated load balancing in peer-to-peer massively multiplayer online role-playing games using hierarchical area-of-interest management. The proposed approach dynamically adjusted the allocation of workers according to player distribution in different regions of the game world. The study demonstrated the importance of load balancing when scaling multiplayer environments [9].

Liu, Claypool, Kuwahara, Scovell, and Sherman studied the effects of network latency on competitive first-person shooter players. Their user study showed that reductions in network latency can improve player performance and quality of experience, while latency compensation can help reduce the negative effects of network delay [10].

The reviewed studies demonstrate that multiplayer gaming involves several interconnected challenges, including network latency, synchronization, consistency, fairness, and scalability. Existing research has proposed distributed architectures and specialized synchronization or latency-compensation techniques to address these challenges. However, these approaches often focus on individual networking problems or large-scale distributed infrastructures.

In contrast, the proposed work focuses on developing a practical and modular multiplayer gaming platform using Unity and Unity Gaming Services. The system combines player authentication, session creation and joining, networked player objects, player ownership, multiplayer services, and real-time gameplay within a

single development framework. A 2D multiplayer platformer is used as the initial implementation and serves as a foundation for extending the architecture to additional multiplayer games. The project therefore demonstrates the practical integration of modern multiplayer development components while providing a reusable foundation for future expansion.d

3 PROPOSED SYSTEM

This section describes the design and implementation of the proposed online multiplayer gaming platform using Unity and Unity Gaming Services. The developed prototype focuses on a 2D multiplayer platformer and establishes a reusable multiplayer structure that can be extended to additional games in the future. The implementation combines player authentication, multiplayer session management, networked player objects, player ownership, input handling, and real-time movement synchronization.

3.1 Overall System Architecture

The proposed platform is developed using Unity as the game engine and Unity Gaming Services (UGS) for multiplayer-related cloud services. Netcode for GameObjects is used to manage networked game objects and communication between connected players. The NetworkManager acts as the central networking component responsible for managing the multiplayer connection and networked objects.

The system follows a host-client architecture. A player can create a multiplayer session and act as the host, while other players can join the same session as clients. Player authentication is performed before participating in multiplayer gameplay. After a session is created, a session code can be used by other players to join the same multiplayer environment.

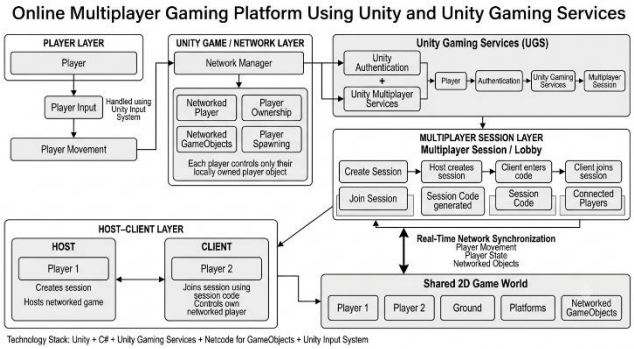


Fig. 1. Overall architecture of the proposed online multiplayer gaming platform.

The major components of the system include Unity, Unity Gaming Services, player authentication, multiplayer session management, NetworkManager, Netcode for GameObjects, Unity Input System, and networked player objects. These components work together to provide a common framework for multiplayer gameplay.

3.2 Unity Gaming Services Initialization and Authentication

Unity Gaming Services is initialized when the application starts so that the required multiplayer services can be accessed. Player authentication is then performed to provide an identity for each participant in the multiplayer environment. Authentication forms an important part of the session workflow because players must be identified before participating in a multiplayer session. The authentication component is integrated into the Unity project along with the services initialization process.

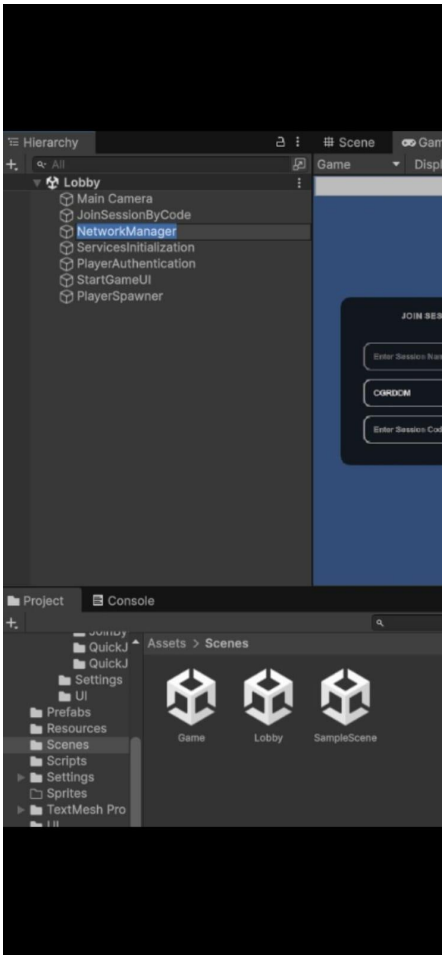


Fig. 2. Unity scene components used for services initialization and player authentication.

3.3 Multiplayer Session Creation and Joining

The multiplayer platform uses a session-code-based mechanism for connecting players. A player can create a multiplayer session, after which the session can be joined by other players using the provided session code.

The joining process allows players to enter the same multiplayer environment without requiring manual network configuration. The session management functionality is integrated with Unity Multiplayer Services and provides the connection mechanism used by the developed 2D multiplayer platformer.

The overall session process can be represented as follows:

Player Authentication → Session Creation/Joining → Player Connection → Player Spawning → Multiplayer Gameplay

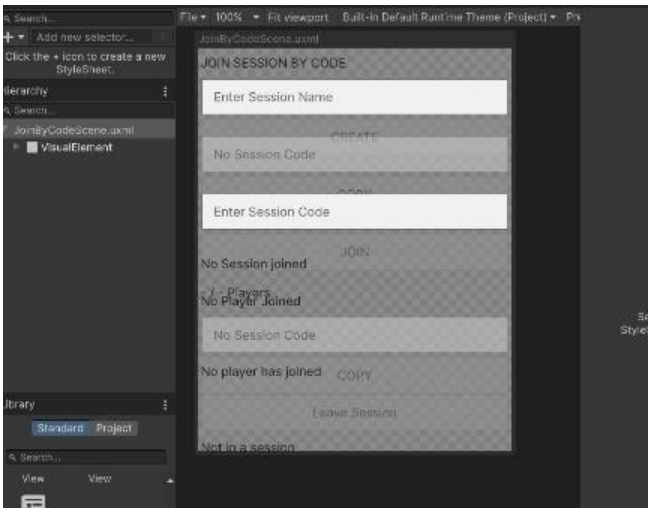


Fig. 3. Session creation and joining workflow of the multiplayer platform.

The session workflow provides the basic connection layer required before gameplay begins. It also establishes the foundation for supporting additional multiplayer games within the platform.

3.4 Networked Player Architecture

Networked player objects are managed using Netcode for GameObjects. When players successfully join the multiplayer session, their corresponding player objects are spawned within the game environment.

Player ownership is used to determine which connected player is responsible for controlling a particular player object. This prevents one player from directly controlling another player's character. The ownership mechanism is therefore important for maintaining independent player control while allowing all connected clients to observe the networked game state.

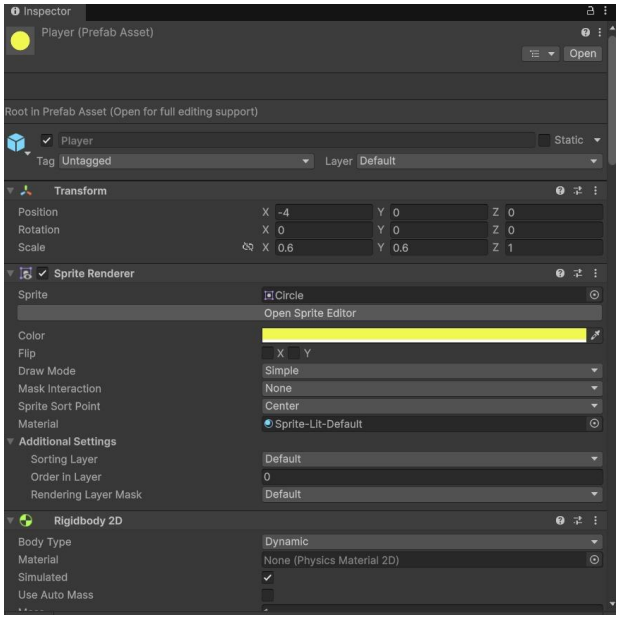


Fig. 4. Networked player prefab and components used for multiplayer player management.

The networked player contains the required components for player input, movement, physics, and network communication. The NetworkObject component allows the player to participate in the networked environment, while ownership identifies the client associated with the player object.

3.5 Player Input And Movement

Player movement is implemented using Unity's Input System. Movement input is captured from the local player and applied to the corresponding networked player object. The input configuration uses movement actions and Vector2 input to support two-dimensional player movement.

During development, input configuration and ownership were tested because a networked player may appear correctly on another client while still being unable to receive the appropriate local input. These issues were investigated by checking Input Actions, player ownership, spawning, and the relationship between the local player and its networked object.

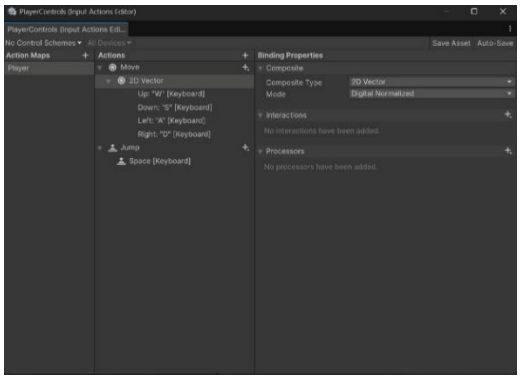


Fig. 5. Player input configuration used for movement in the 2D multiplayer environment.

The movement implementation allows each player to control their own character while the networked system communicates the relevant player state to other connected participants.

3.6 2D Multiplayer Game Environment

The initial game developed on the proposed platform is a 2D multiplayer platformer. The environment contains a ground area, multiple platforms, player objects, camera components, lighting, and a multiplayer user interface.

The Unity scene contains the NetworkManager and the required services and multiplayer components. The game environment is used to test player spawning, movement, ownership, and interaction between multiple connected players.

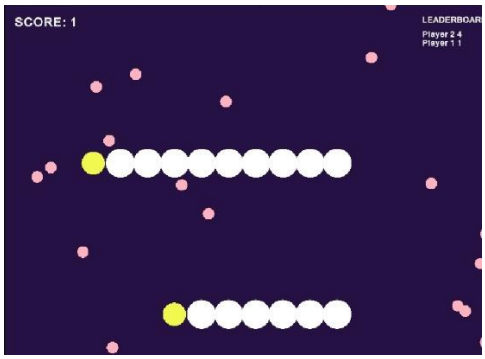


Fig. 6. Developed 2D multiplayer platformer environment.

The platformer serves as the initial implementation of the proposed multiplayer framework. Its purpose is to demonstrate that the common networking components can be integrated into an interactive multiplayer game environment.

3.7 Multiplayer Gameplay and Synchronization

After players join the same session, their networked player objects are created in the shared game environment. Each player controls their own character while the other connected players can observe the corresponding networked movement.

Network synchronization is handled through Netcode for GameObjects and the associated networked objects. Player ownership and network object management are used together to distinguish local control from remote player representation.

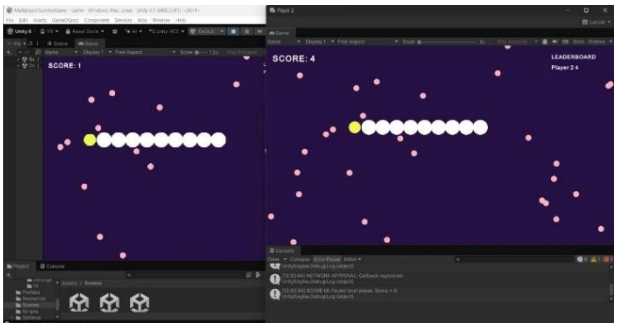


Fig. 7. Two connected players participating in the same multiplayer game session.

The multiplayer gameplay test is an important validation of the proposed architecture because it demonstrates the complete workflow from connection and player spawning to simultaneous gameplay.

3.8 Multiplayer User Interface

A multiplayer user interface is included to provide players with the controls required for session-based gameplay. The interface supports the operations required to create or join multiplayer sessions using a session code.

The user interface is connected with the session management functionality so that players can initiate or participate in multiplayer gameplay through the application.

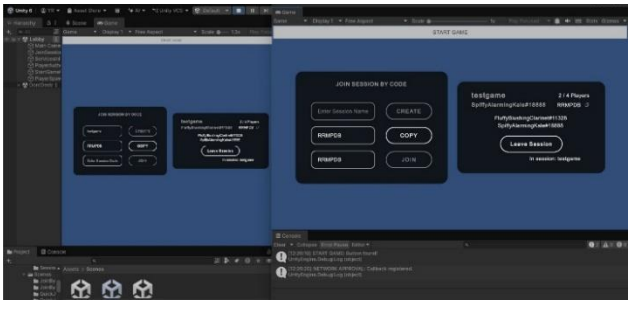


Fig. 8. User interface for creating and joining multiplayer sessions.

3.9 Multiplayer Lobby System

A multiplayer lobby system is implemented to manage players before the start of multiplayer gameplay. After authentication, players can enter the multiplayer lobby and participate in the session created by the host. The lobby provides a controlled environment for managing connected players before they enter the game.

The lobby system is integrated with the multiplayer session management component. Players can create or join a session and wait for other participants before starting the game. This provides an organized transition between session joining and actual multiplayer gameplay.



Fig. 9. Multiplayer lobby interface showing connected players before gameplay.

The lobby system improves the multiplayer experience by providing a dedicated stage for player connection and session preparation. It also provides a foundation for extending the platform with additional lobby features such as player status, ready states, and matchmaking.

3.10 Player Elimination System

A player elimination system is implemented to manage player removal during multiplayer gameplay. The system monitors the player's gameplay state and performs the required elimination operation when the player reaches the defined elimination condition.

When a player is eliminated, the corresponding player state is updated and the elimination is reflected in the multiplayer environment. The system is integrated with the networked player architecture so that the elimination state can be observed by the connected players.

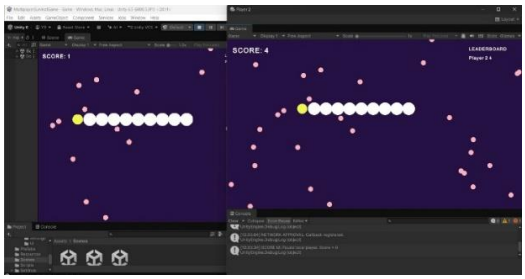


Fig. 10. Player elimination functionality during multiplayer gameplay.

The elimination mechanism provides a basic gameplay rule for multiplayer competition and can be further extended with respawning, health systems, game rounds, and victory conditions.

3.11 Leaderboard System

A leaderboard system is implemented to display and compare player performance during multiplayer gameplay. The leaderboard maintains player scores and provides a ranking of participants based on their accumulated performance.

The leaderboard is integrated with the multiplayer gameplay and scoring components so that changes in player performance can be reflected in the displayed rankings. This allows players to compare their results with other participants.

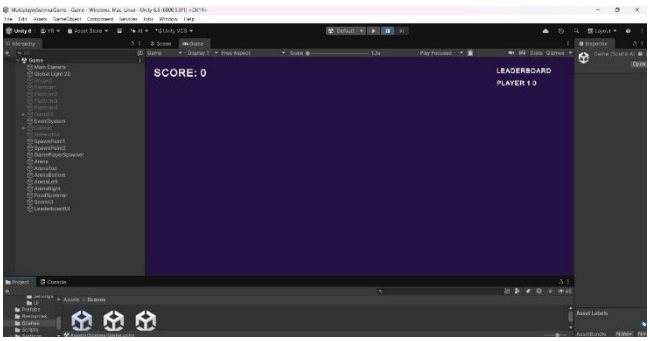


Fig. 11. Leaderboard displaying player scores and rankings.

The leaderboard provides a competitive element to the multiplayer platform and establishes a basis for future extensions such as persistent player statistics, achievements, and online rankings.

3.12 Development and Debugging

The development process involved testing the interaction between Unity's input system, networked objects, player ownership, session management, player spawning, lobby management, elimination, and leaderboard functionality. Several networking-related issues were investigated during implementation.

Input Action configuration and Vector2 movement bindings were checked to ensure that player input was correctly received. Player ownership was also tested because the second player's object could be visible without receiving the expected local control. Networked object spawning and player lifecycle behavior were examined to ensure that connected players were correctly represented in the game.

The lobby, elimination, and leaderboard components were also tested as part of the multiplayer gameplay workflow. Session lifecycle behavior was examined to ensure that players could enter

and leave the appropriate stages of the multiplayer session without performing invalid session operations.

These debugging and testing activities helped improve the reliability and stability of the multiplayer prototype.

3.13 3D Multiplayer Racing Module

The 3D multiplayer racing game is implemented using Unity and Netcode for GameObjects. The racing environment contains a 3D track with roads, barriers, fences, cones, tyres, finish-line structures, signs, stands, lighting, advertising boards, and other environment objects. A dedicated racing scene is used to manage the track and racing components.

The racing system uses a RaceManager to initialize the race and create networked racing vehicles for connected players. Three predefined vehicle spawn positions are provided so that each player can start from a separate position on the track. Each racing vehicle is configured as a NetworkObject, and ownership is assigned to the corresponding player.

Vehicle control follows a server-authoritative networking model. The local client collects driving input and sends the input values to the server through RPC communication. The server processes the received input and controls the vehicle Rigidbody. The vehicle position and rotation are then synchronized across the connected clients using NetworkTransform.

The vehicle supports forward and reverse movement using throttle input and left and right steering. A maximum speed is applied to control vehicle movement. Rigidbody physics is used for movement and collision handling, while the server maintains authority over the vehicle's physical state.

A third-person follow camera is provided for each player. The camera identifies the locally owned racing vehicle and follows it using a configured offset and smooth movement. This allows the player to view and control their own vehicle while observing other networked vehicles on the same track.

A synchronized race countdown is used before the race begins. The server controls the countdown sequence as 3, 2, 1, and GO. Vehicle movement remains disabled during the countdown. After the GO state, the race starts and vehicle input is enabled for the players.

The racing track contains multiple checkpoints implemented using trigger colliders. Each checkpoint has a unique number, and the system maintains the next checkpoint expected by each vehicle. Checkpoints must be passed in the defined order. For example, when the sequence is CP1 → CP2 → CP3, passing CP3 before CP2 does not increase the player's progress.

The checkpoint system controls lap progression. Three checkpoints form one complete lap. After successfully passing CP1, CP2, and CP3, the next expected checkpoint is reset to CP1 and the vehicle's lap count is increased. Each vehicle maintains its own current lap, next checkpoint, and race progress, allowing different players to progress independently.

The race is configured for five laps. The race management system identifies completion after the required lap progression is reached. The racing implementation therefore combines networked vehicle spawning, ownership, server-authoritative movement, synchronized race initialization, checkpoint validation, and multi-lap race progression within the multiplayer game environment.

3.14 Overall Development Workflow

The complete development workflow can be summarized as: **Unity Project Initialization → UGS Initialization → Player Authentication → Session Creation/Joining → Lobby → Network Connection → Player Object Spawning → Ownership Assignment → Game Selection → 2D Platformer / 3D Racing → Input and Game-Specific Control → Network Synchronization → Multiplayer Gameplay → Elimination and Score Updates → Leaderboard Display → Multiplayer Gameplay Testing**

The 3D racing flow includes networked vehicle spawning, vehicle ownership assignment, server-authoritative vehicle control, synchronized race countdown, checkpoint validation, and multi-lap race progression. For the racing mode, the connected players are assigned networked vehicles after entering the racing environment. Player input is sent to the server, vehicle movement is processed by the server, and the resulting vehicle state is synchronized with the connected clients. Checkpoint and lap states are maintained for each racing vehicle during the race.



```
1 reference
private void SpawnCars()
{
    if (raceStarted)
        return;

    raceStarted = true;

    int playerIndex = 0;

    foreach (ulong clientId in NetworkManager.Singleton.ConnectedClientsIds)
    {
        if (playerIndex >= spawnPoints.Length)
        {
            Debug.LogWarning("More players than available race spawn points.");
            break;
        }

        Transform spawnPoint = spawnPoints[playerIndex];

        GameObject car = Instantiate(
            carPrefab,
            spawnPoint.position,
            spawnPoint.rotation
        );

        NetworkObject networkCar = car.GetComponent<NetworkObject>();

        if (networkCar == null)
        {
            Debug.LogError("Microcar_Blue does not have a NetworkObject!");
            Destroy(car);
            continue;
        }

        networkCar.SpawnWithOwnership(clientId);

        Debug.Log(
            "Spawned racing car for Client " +
            clientId +
            " at SpawnPoint " +
            (playerIndex + 1)
        );

        playerIndex++;
    }
}
```

Fig. 12. Overall development and multiplayer gameplay workflow.

The resulting system integrates Unity game development, Unity Gaming Services, player authentication, session management, lobby management, real-time networking, networked object synchronization, player ownership, gameplay management, player elimination, scoring, leaderboard functionality, and multiple game modes. The platform supports both the 2D multiplayer platformer and the 3D multiplayer racing game within the same multiplayer environment. The common multiplayer workflow handles the connection and network management, while each game mode manages its respective gameplay operations such as player movement and elimination in the platformer and vehicle control, checkpoints, and lap progression in the racing game.

4 EVALUATION

The developed multiplayer gaming prototype was tested to verify the functionality of player authentication, session management, networked player spawning, player ownership, input handling, and multiplayer movement synchronization. Testing was performed using multiple game instances connected to the same multiplayer session.

4.1 Authentication and Session Creation

The authentication and multiplayer session workflow was tested before starting multiplayer gameplay. A player was able to initialize the multiplayer services, authenticate, and create a multiplayer session. The created session generated a session code that could be provided to another player.

```
SessionSettings m_SessionSettings;

CreateSessionViewModel m_ViewModel;

readonly List<DataBinding> m_Bindings = new();

1 reference
public CreateSessionElement()
{
    AddToClassList(BlocksTheme.ContainerHorizontal);

    var enabledBinding = new DataBinding
    {
        dataSourcePath = new PropertyPath(nameof(m_ViewModel.CanRegisterSession)),
        bindingMode = BindingMode.ToTarget
    };
    SetBinding(new BindingId(nameof(enabledSelf)), enabledBinding);
    m_Bindings.Add(enabledBinding);

    var sessionNameTextField = new TextField
    {
        textEdition =
        {
            placeholder = k_EnterSessionNamePlaceholder,
            hidePlaceholderOnFocus = true
        }
    };
    sessionNameTextField.AddToClassList(BlocksTheme.TextField);
    sessionNameTextField.AddToClassList(BlocksTheme.SpaceRight);
    var sessionNameBinding = new DataBinding
```

Fig. 13 Successful multiplayer session creation and session code generation.

The session code provided a simple mechanism for connecting players to the same multiplayer environment without requiring manual network configuration.

4.2 Session Joining

A second player was tested by entering the session code generated by the host. The client successfully joined the existing multiplayer session and was connected to the same game environment.

Successful session joining confirms that the session management component can establish a shared multiplayer environment between the host and client.

4.3 Networked Player Spawning

After the client joined the session, the corresponding networked player object was spawned in the game environment. Both the host and client could observe the connected players within the same 2D platformer scene.

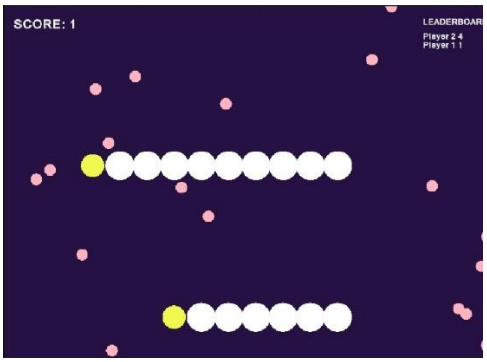


Fig. 14. Two networked players spawned in the same multiplayer

4.4 Player Movement and Synchronization

Player movement was tested using two connected players. Each player was able to control the corresponding local player character, while the movement of the other player was visible in the connected game instance.

The test confirms the correct interaction between the Unity Input System, player ownership, networked player objects, and Netcode for GameObjects. The local player receives its input while the networked representation of the player is made available to the other connected participant.

4.5 Player Ownership Testing

Player ownership was tested to ensure that each connected player controlled the appropriate player object. The tests showed

that ownership is required to distinguish local player control from remote player representation.

During development, an issue was observed in which the second player's object could be visible but did not initially respond to the expected input. Investigation of the Input System configuration, player ownership, and networked player setup helped identify the cause and improve the player control behavior.

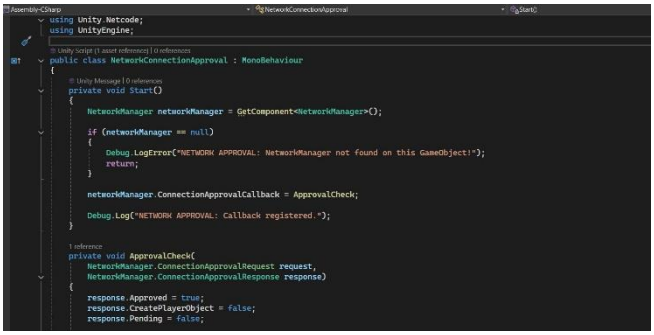


Fig. 15. Networked player configuration used for player ownership and local control.

4.6 Multiplayer Lobby Testing

The implemented multiplayer lobby system was tested to verify that connected players could be managed before entering the gameplay environment. After authentication and session creation or joining, players could enter the multiplayer lobby and be represented as participants in the same multiplayer session.

The lobby provides an intermediate stage between session connection and gameplay, allowing the connected players to be organized before the game begins. The functionality was tested with multiple game instances to verify that the connected players were correctly recognized within the multiplayer session.

The successful lobby test demonstrates that the platform can manage connected participants before starting multiplayer gameplay. The lobby also provides a foundation for future improvements such as ready-state management and advanced matchmaking.

4.7 Player Elimination Testing

The player elimination system was implemented and tested as part of the multiplayer gameplay. The system manages the elimination of a player when the defined elimination condition is reached during the game.

During testing, the elimination behavior was observed to ensure that the appropriate player's gameplay state was updated and that the elimination was reflected in the multiplayer environment. The system was also tested with connected players to verify that the elimination event could be observed across the multiplayer session.

The test demonstrates that the elimination mechanism can be integrated with the networked player architecture and multiplayer gameplay. This feature adds a competitive gameplay element to the developed 2D multiplayer platformer.

4.8 Leaderboard Testing

The implemented leaderboard system was tested to verify the display and comparison of player performance during multiplayer gameplay. Player scores were recorded and displayed according to their performance in the game.

The leaderboard provides a common view of player rankings and allows connected participants to compare their scores. The functionality was tested during multiplayer gameplay to verify that player results were correctly reflected in the leaderboard.

The successful leaderboard test demonstrates that the platform can incorporate player performance tracking into multiplayer gameplay. The feature provides a competitive element and can be further extended with persistent statistics and achievements.

4.9 Multiplayer Gameplay Test

The complete multiplayer workflow was tested by running two players in the same session. The host created the session and the client joined using the session code. The connected players entered the multiplayer lobby before participating in the game.

Both player objects were successfully displayed in the shared game environment, and each player was able to control the corresponding character. Player movement could be observed across the connected game instances. The gameplay test also included the implemented elimination and leaderboard functionality.

The complete test demonstrates the integration of authentication, session management, lobby functionality, networked player spawning, player ownership, movement synchronization, elimination, and leaderboard functionality within the developed multiplayer platform.

4.10 Session Lifecycle and Network Testing

The multiplayer session lifecycle was also examined during development. Session creation, joining, lobby participation, player spawning, networked gameplay, elimination, and leaderboard updates were tested as connected stages of the multiplayer workflow.

Session-related issues were investigated to avoid performing operations when a corresponding multiplayer session was not active. The testing process also included investigation of player spawning, networked object behavior, input configuration, ownership, lobby behavior, elimination, and leaderboard functionality.

These tests were important for improving the reliability and stability of the developed multiplayer prototype.

4.11 3D Racing Module Evaluation

The 3D multiplayer racing module was evaluated to verify networked vehicle spawning, vehicle ownership, server-authoritative vehicle movement, input transmission, race synchronization, checkpoint validation, and multi-lap progression. The evaluation was performed using multiple connected game instances participating in the same racing session.

4.11.1 Racing Environment and Vehicle Spawning

The racing scene was tested to verify the initialization of the 3D track environment and the creation of networked racing vehicles. The scene contains the racing track, road boundaries, barriers, cones, finish-line structures, signs, lighting, and other environment components.

The RaceManager creates racing vehicles for connected players when the racing scene is active. Three predefined spawn positions are available for the vehicles, allowing connected players to start from separate positions on the track. Each vehicle is configured as a NetworkObject and is associated with the corresponding connected player.



Fig. 16. Networked racing vehicles spawned for connected players.

4.11.2 Vehicle Ownership and Input Testing

Vehicle ownership was tested to verify that each player could control only the racing vehicle assigned to that player. The local client receives control of its owned vehicle, while other vehicles remain visible as networked objects.

The vehicle input system was tested using throttle and steering controls. Input values are transmitted from the client to the server through RPC communication rather than directly modifying the final vehicle position. This allows the server to process the received input and maintain the authoritative vehicle state.

4.11.3 Server-Authoritative Vehicle Movement

The vehicle movement was evaluated to verify server-side processing of driving input and synchronization of the resulting vehicle state. Forward and reverse movement, steering, maximum speed, and Rigidbody-based movement were tested during the race.

The server applies the received input to the vehicle Rigidbody, while NetworkTransform synchronizes the vehicle position and rotation with the connected clients. This allows all participants to observe the movement of the networked vehicles within the shared racing environment.

```
[Rpc(SendTo.Server)]
1 reference
private void SendInputRpc(float throttle, float steer)
{
    moveThrottle = throttle;
    moveSteer = steer;
}

Unity Message | 0 references
private void FixedUpdate()
{
    // Physics is controlled by the server.
    if (!IsServer)
        return;

    MoveCar();
    SteerCar();
}

1 reference
private void MoveCar()
{
    Vector3 velocity = rb.linearVelocity;

    // Keep the current vertical velocity so gravity still works.
    float verticalVelocity = velocity.y;

    if (Mathf.Abs(moveThrottle) > 0.01f)
    {
        float speed = moveThrottle > 0f
            ? maxSpeed
            : reverseSpeed;

        Vector3 targetVelocity =
            transform.forward *
            (moveThrottle * speed);
```

Fig. 17. Server-authoritative vehicle movement and network synchronization.

4.11.4 Race Countdown and Movement Lock Testing

The race countdown was tested to verify that all connected players receive a synchronized race start. The server controls the countdown sequence using 3, 2, 1, and GO.



Fig. 18. Synchronized racing countdown before vehicle movement.

During the countdown, vehicle input remains disabled through the race-start state. After the GO state is reached, the race is marked as started and vehicle control is enabled. This prevents players from starting the race before the synchronized starting point.

4.11.5 Checkpoint Validation Testing

The checkpoint system was tested to verify that racing vehicles must pass checkpoints in the specified order. Each checkpoint contains a trigger collider and a unique checkpoint number.

The evaluation used the checkpoint sequence CP1 → CP2 → CP3. A vehicle successfully advances when it reaches the expected checkpoint. If a player attempts to pass a later checkpoint before completing the required checkpoint, the race progress is not advanced. Repeatedly triggering the same checkpoint also does not incorrectly increase the player's progress.

4.11.6 Multi-Lap Race Testing

The multi-lap system was evaluated using three checkpoints as one complete lap. After a vehicle successfully passes CP1, CP2, and CP3, the expected checkpoint is reset to CP1 and the vehicle's lap count is increased.

Each racing vehicle maintains its own current lap, next expected checkpoint, and total race progress. Therefore, connected players can be at different checkpoints or laps while participating in the same race. The configured racing test uses five laps for race completion.



Fig. 19. Multi-lap race progression with individual player race states.

4.11.7 Racing Module Evaluation

The complete racing workflow was evaluated from vehicle initialization through race progression. The test sequence included networked vehicle spawning, ownership assignment, client input transmission, server-side vehicle movement, transform synchronization, synchronized countdown, checkpoint validation, and lap progression.

The evaluation confirms that the racing components operate within the multiplayer environment while maintaining independent race states for individual vehicles. The current implementation successfully provides networked vehicle control, synchronized race initialization, ordered checkpoint processing, and multi-lap progression. Final finishing-position calculation, racing leaderboard integration, car selection, and track selection are not included in the current implementation.

5 CONCLUSION AND FUTURE WORK

5.1 Conclusion

The developed system provides a reusable real-time multiplayer gaming platform implemented using Unity and Unity Gaming Services. The platform integrates authentication, session creation and joining, lobby management, networked player spawning, ownership, input handling, movement synchronization, elimination, and leaderboard functionality within a common multiplayer structure. The implementation supports both a 2D multiplayer platformer and a 3D multiplayer racing game through the same underlying multiplayer workflow.

The 3D racing implementation demonstrates networked vehicle spawning, ownership assignment, server-authoritative vehicle movement, RPC-based input transmission, synchronized countdown, checkpoint validation, and multi-lap race progression. The racing module currently supports a three-checkpoint lap structure and a five-lap race configuration, with race completion detection implemented.

Overall, the implementation establishes a common multiplayer foundation while allowing different gameplay mechanics to operate within the same platform. The system can be further developed by extending game modes, improving race result handling, and adding additional multiplayer features.

5.2 Future Work

The developed platform can be further extended to provide a more complete and scalable online multiplayer gaming environment. Future development can focus on introducing multiple levels, additional gameplay variations, and power-ups to provide greater game variety and player interaction.

The proposed multiplayer architecture can be reused to develop additional multiplayer mini-games, allowing different games to operate using the same authentication, session management, lobby, and networking framework. The platform can also be enhanced with advanced lobby features such as player lists, ready states, matchmaking, and private game rooms.

For the 3D racing game, future improvements can include lap-progress display, final finishing-position calculation, racing-specific leaderboard functionality, car selection, and track selection.

Further development may include player profiles, statistics, achievements, improved handling of player disconnections and reconnections, and large-scale multiplayer testing. Performance evaluation with more concurrent players can be conducted to study network synchronization, latency, and scalability. Cross-platform support can also be explored to allow players using different supported devices to participate in the same multiplayer environment.

REFERENCES

[1] [1] L. Gautier and C. Diot, "Design and evaluation of MiMaze, a multi-player game on the Internet," in *Proc. 3rd IEEE Multimedia Computing and Systems (MMCS)*, 1998, pp. 233–236, doi: 10.1109/MMCS.1998.693647.

[2] [2] Y.-J. Lin, K. Guo, and S. Paul, "Sync-MS: Synchronized messaging service for real-time multi-player distributed games," in *Proc. 10th IEEE Int. Conf. Network Protocols (ICNP)*, 2002, pp. 155–164, doi: 10.1109/ICNP.2002.1181396.

[3] [3] X. Jiang, F. Safaei, and P. Boustead, "Latency and scalability: A survey of issues and techniques for supporting networked games," in *Proc. 13th IEEE Int. Conf. Networks (ICON)*, 2005, vol. 1, doi: 10.1109/ICON.2005.1635458.

[4] [4] K. S. Moon, V. Muthukkumarasamy, A. T.-A. Nguyen, and H. S. Kim, "Maintaining consistency in distributed network games," in *Proc. 13th IEEE Int. Conf. Networks (ICON)*, 2005, vol. 1, doi: 10.1109/ICON.2005.1635506.

[5] [5] M. Mauve, J. Vogel, V. Hilt, and W. Effelsberg, "Local-lag and timewarp: Providing consistency for replicated continuous applications," *IEEE Trans. Multimedia*, vol. 6, no. 1, pp. 47–57, Feb. 2004, doi: 10.1109/TMM.2003.819751.

[6] [6] L. Pantel and L. C. Wolf, "On the impact of delay on real-time multiplayer games," in *Proc. 12th Int. Workshop Network and Operating Systems Support for Digital Audio and Video (NOSSDAV)*, 2002, pp. 23–29, doi: 10.1145/507670.507674.

[7] [7] Y. W. Ahn, A. M. K. Cheng, J. Baek, and P. S. Fisher, "A multiplayer real-time game protocol architecture for reducing network latency," *IEEE Trans. Consumer Electron.*, vol. 55, no. 4, pp. 1883–1889, Nov. 2009, doi: 10.1109/TCE.2009.5373746.

[8] [8] A. Bharambe, J. Pang, and S. Seshan, "Colyseus: A distributed architecture for online multiplayer games," in *Proc. 3rd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, San Jose, CA, USA, 2006, pp. 155–168.

[9] [9] S. Fujita, "Load balancing of peer-to-peer MMORPG systems with hierarchical area-of-interest management," *Int. J. Networked and Distributed Computing*, vol. 3, no. 3, pp. 177–184, Aug. 2015, doi: 10.2991/ijndc.2015.3.3.5.

[10] [10] S. Liu, M. Claypool, A. Kuwahara, J. Scovell, and J. Sherman, "The effects of network latency on competitive first-person shooter game players," in *Proc. 13th Int. Conf. Quality of Multimedia Experience (QoMEX)*, 2021, pp. 151–156, doi: 10.1109/QoMEX51781.2021.9465419.